

Venus fixed-rate vault Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/04/20 - 2026/04/23 (YYYY/MM/DD)

Overall Risk: **Medium**

Project Name	Venus fixed-rate vault Audit
Github	https://github.com/VenusProtocol/fixed-rate-vaults/pull/2 Fix: https://github.com/VenusProtocol/fixed-rate-vaults/pull/4
Background	Can I check if there would be capacity to kickoff another concurrent audit? 🙏 https://www.notion.so/Audit-scope-Fixed-Rate-Vaults-Institutional-Vaults-2026-04-16-344b4e5771bb80749aa1f3497eaccc76?source=copy_link Fix: https://www.notion.so/185-34fb4e5771bb80e7a644e41b1a45bca4

Smart contracts list:

No.	File Name	Link	Verdict	Details
1	BaseVault.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/BaseVault.sol	Low	[L01]-Ack [L02]-Ack [I02]-Ack [I04]-Fixed
2	InstitutionalLoanVault.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/institutional-vault/InstitutionalLoanVault.sol	Medium	[M01]-Fixed [M02]-Fixed [I01]-Fixed [I04]-Fixed [I05]-Fixed [I06]-Ack
3	InstitutionalVaultController.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/institutional-vault/InstitutionalVaultController.sol	Medium	[M02]-Fixed [L02]-Ack [I03]-Fixed [I04]-Fixed
4	LiquidationAdapter.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/institutional-vault/LiquidationAdapter.sol	Informational	[I02]-Ack
5	InstitutionPositionToken.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/institutional-vault/InstitutionPositionToken.sol	Green	
6	IVaultTypes.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IVaultTypes.sol	Green	
7	IIInstitutionalVaultTypes.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IIInstitutionalVaultTypes.sol	Green	

8	IVaultController.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IVaultController.sol	Green	
9	IIstitutionalVaultController.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IIstitutionalVaultController.sol	Green	
10	IIstitutionalLoanVault.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IIstitutionalLoanVault.sol	Green	
11	IIstitutionPositionToken.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IIstitutionPositionToken.sol	Green	
12	ILiquidationAdapter.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/ILiquidationAdapter.sol	Green	
13	IProtocolShareReserve.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IProtocolShareReserve.sol	Green	
14	IResilientOracle.sol	https://github.com/VenusProtocol/fixed-rate-vaults/blob/feat/vpd-774-institutional-vault/src/interfaces/IResilientOracle.sol	Green	

Findings

[M01] depositCollateral in WaitingForMargin Requires the Full Margin in a Single Transaction, Contradicting NatSpec

Contract InstitutionalLoanVault.sol

Severity Level Medium

Description

Description

In WaitingForMargin state, depositCollateral() transfers tokens and increments totalCollateralDeposited before checking whether the cumulative total meets the margin threshold. If it does not, the entire transaction reverts — including the transfer — so no partial deposit ever persists on-chain. The institution must therefore deposit the full margin amount in a single transaction. The NatSpec explicitly states "cumulative deposits must reach margin amount to transition to MarginDeposited", implying that incremental deposits across multiple transactions should be supported.

JavaScript

```
// InstitutionalLoanVault - depositCollateral()
collateralToken.safeTransferFrom(msg.sender, address(this), amount);
uint256 actual = collateralToken.balanceOf(address(this)) - balanceBefore;
_instRuntime.totalCollateralDeposited += actual;

if (s == VaultState.WaitingForMargin) {
    uint256 marginAmount = (_instConfig.idealCollateralAmount *
_instConfig.marginRate) / MANTISSA_ONE;
    if (_instRuntime.totalCollateralDeposited < marginAmount) revert
InsufficientCollateral();
    _stateTransition(VaultState.MarginDeposited);
}
```

Impact:

If the institution holds funds across multiple wallets, or if its treasury system imposes per-transaction transfer limits, it cannot complete the WaitingForMargin → MarginDeposited transition. Every new vault's onboarding flow is affected.

Recommendation Replace the revert with a conditional transition so partial deposits are always preserved:

JavaScript

```
if (s == VaultState.WaitingForMargin) {
    uint256 marginAmount = (_instConfig.idealCollateralAmount *
_instConfig.marginRate) / MANTISSA_ONE;
    if (_instRuntime.totalCollateralDeposited >= marginAmount) {
        _stateTransition(VaultState.MarginDeposited);
    }
}
```

```
}  
}
```

Status

Fixed in [a3502cb](#). The development team confirmed that single-transaction margin deposit is intentional — the margin amount is expected to be small enough to deposit in one transaction. The fix updates the NatSpec on `depositCollateral` to align with the actual code behaviour, resolving the contradiction.

JavaScript

```
// InstitutionalLoanVault - depositCollateral()  
/// @notice Deposits collateral into the vault.  
/// - WaitingForMargin: the full margin amount must be  
deposited in a single transaction  
/// - to transition to MarginDeposited; partial deposits  
revert.  
/// - Fundraising: institution deposits remaining collateral  
alongside lender fundraising.  
/// - Lock: top-up collateral.
```

[M02] Missing $LI \times LT < 1$ Invariant Validation Allows Parameter Combinations Where Each Liquidation Worsens Vault Health

Contract InstitutionalVaultController.sol, InstitutionalLoanVault.sol

Severity Level **Medium**

Description

Description

The controller validates `liquidationThreshold` (LT) and `liquidationIncentive` (LI) independently but never checks their product. The post-liquidation shortfall change is derived directly from the contract's code:

JavaScript

```
shortfall_new = shortfall_old + repayUSD * (LI * LT - 1)
```

When $LI \times LT > 1$, each liquidation increases the shortfall. This formula follows directly from `_receiveRepayment` reducing debt by `repayUSD`, and `_calculateSeizeAmount` removing collateral worth `repayUSD × LI`, substituted into `shortfall = debtUSD - collateralUSD × LT`.

JavaScript

```
// InstitutionalVaultController - _validateVaultConfig() / setters
if (newLT == 0 || newLT > MANTISSA_ONE) revert ...;
if (newLI <= MANTISSA_ONE || newLI > MANTISSA_ONE_AND_HALF) revert ...;
// LI × LT product is never validated
```

Valid combinations such as $LT = 0.95e18$ and $LI = 1.1e18$ yield $LI \times LT = 1.045e18 > 1$.

Impact:

With $LI \times LT > 1$, liquidators extract disproportionately large collateral incentives per unit of debt repaid. Collateral is depleted faster than debt is reduced until `InsufficientCollateralForSeize` halts further liquidation. The remaining debt becomes bad debt absorbed by lenders, who recover significantly less than they would under correctly configured parameters. Requires governance to configure the problematic parameter combination.

Recommendation Add a product check in `_validateVaultConfig` and all three risk parameter setters:

JavaScript

```
if (riskConfig.liquidationIncentive * riskConfig.liquidationThreshold >=
MANTISSA_ONE * MANTISSA_ONE) {
    revert InvalidConfig();
}
```

The same constraint applies to `latePenaltyRate`: a $\text{latePenaltyRate} \times LT$ product exceeding 1 causes the overdue liquidation path to worsen vault health in the same way, so `_validateVaultConfig` must also validate $\text{latePenaltyRate} \times \text{liquidationThreshold} < MANTISSA_ONE^2$, and all three setters must cross-validate both products against the current live values when any single parameter is updated.

Status

Fixed in [84af7cb](#). A dedicated `_validateLiquidationInvariant` helper was added to `InstitutionalVaultController`. The invariant is now enforced at vault creation time in `_validateVaultConfig`, and is re-checked in all three risk parameter setters whenever any single parameter is updated — using the vault's live `riskConfig()` to cross-validate against current values.

JavaScript

```
// InstitutionalVaultController - _validateLiquidationInvariant
/// @dev Reverts if `riskFactor * lt` reaches 1e36 (i.e. >= 1.0 in
mantissa).
/// A product >= 1.0 means liquidations would worsen vault health
rather than improve it.
```

```

    ///      Used for both `LI * LT` and `latePenaltyRate * LT` checks.
    function _validateLiquidationInvariant(uint256 lt, uint256 riskFactor)
private pure {
    if (riskFactor * lt >= MANTISSA_ONE * MANTISSA_ONE) revert
InvalidConfig();
}

```

[L01] sweep() Has No State Restriction or Token Exclusion, Allowing Governance to Extract Core Assets from Active Vaults

Contract BaseVault.sol

Severity Level Low

Description **Description**
sweep() is only guarded by onlyController with no vault lifecycle state check and no exclusion for the vault's supplyAsset or collateralAsset. A governance-authorized caller can invoke it at any point — including during Fundraising or Lock — and transfer the entire balance of any token to treasury.

Related Code

```

JavaScript
// BaseVault - sweep()
function sweep(address token) external onlyController {
    uint256 amount = IERC20(token).balanceOf(address(this));
    if (amount == 0) revert NothingToSweep();
    address recipient = IVaultController(vaultController).treasury();
    IERC20(token).safeTransfer(recipient, amount);
    emit TokensSwept(token, recipient, amount);
}

```

Impact:

Sweeping supplyAsset during Fundraising transfers all lender deposits to treasury while _runtime.totalRaised remains unchanged, corrupting all downstream settlement accounting. Sweeping collateralAsset during Lock silently removes the institution's collateral while _instRuntime.totalCollateralDeposited retains its stale value. Requires a malicious or mistaken ACM-authorized governance account.

Recommendation Restrict sweep() from being called with the vault's supplyAsset or collateralAsset, or limit calls to terminal states only (Matured/Failed/Liquidated/Closed).

Status	Acknowledged. The sweep() function remains unchanged. The development team's position is that the lack of token exclusions or state restrictions is intentional — sweep() exists as an emergency governance tool for exploit scenarios, upstream incidents on supply or collateral assets, or migration away from a flawed vault clone, and restricting it would disable the path it exists to enable. This is a reasonable acknowledgement given that sweep() is already gated behind onlyController and the ACM governance process, placing it within the same trust boundary as all other governance-only operations in the protocol.
---------------	--

[L02] closeVault() Has No Outstanding-Shares Check, Allowing Governance to Permanently Lock Lender Funds

Contract	BaseVault.sol, InstitutionalVaultController.sol
Severity Level	Low
Description	Description closeVault() transitions the vault to Closed state without verifying that all shares have been redeemed (totalSupply() == 0) or that settlementAmount == 0. Once Closed, both withdraw() and redeem() revert because their state guards only permit Matured/Failed/Liquidated. The NatSpec warns that governance should only call this after all suppliers have withdrawn, but there is no code-level enforcement. <pre> JavaScript // BaseVault - closeVault() function closeVault() external onlyController { VaultState s = _runtime.state; if (s != VaultState.Matured && s != VaultState.Failed && s != VaultState.Liquidated) revert InvalidState(); _stateTransition(VaultState.Closed); emit VaultClosed(s); } </pre> Impact: Any lender who has not redeemed before closeVault() is called can never recover their funds. Requires a malicious or mistaken ACM-authorized governance account.
Recommendation	Add an outstanding-shares guard with a state-aware grace period override. During the grace period, closeVault() is blocked if any shares remain outstanding, preventing accidental early closure. Once the grace period expires, governance may force-close the vault; any residual assets should be swept to treasury via sweep() and redistributed off-chain. The grace period anchor differs by terminal state: openEndTime for Failed vaults (counted from when fundraising ended) and settlementDeadline for Matured and Liquidated vaults (counted from when the

settlement window closed), ensuring lenders have adequate time to redeem regardless of which path the vault took.

```
JavaScript
// BaseVault - closeVault()
function closeVault() external onlyController {
    VaultState s = _runtime.state;
    if (s != VaultState.Matured && s != VaultState.Failed && s !=
VaultState.Liquidated)
        revert InvalidState();

    uint256 graceStart = (s == VaultState.Failed)
        ? _runtime.openEndTime
        : _runtime.settlementDeadline;

    if (totalSupply() != 0 && block.timestamp < graceStart +
CLOSE_GRACE_PERIOD) {
        revert SharesOutstanding();
    }
    _stateTransition(VaultState.Closed);
    emit VaultClosed(s);
}
```

CLOSE_GRACE_PERIOD should be a protocol-level constant (e.g. 90 days). After the grace period lapses, governance can force-close the vault and use sweep() to recover any remaining supply assets to treasury for off-chain settlement with non-withdrawing lenders.

Status	Acknowledged. The closeVault() function remains unchanged. The development team notes that all preconditions are verified off-chain before the call is made, consistent with the ACM governance process. They also raise a valid concern: a hard totalSupply() ==0 on-chain guard could be permanently blocked by dust shares from rounding or a single non-redeeming lender. This is precisely what the recommended grace period approach was designed to solve — giving lenders adequate time to withdraw while allowing governance to force-close after the period expires. The underlying risk remains: a governance mistake before the off-chain verification step could permanently lock unredeemed lender funds with no on-chain recovery path.
--------	--

[I01] idealCollateralValuation Is Stored but Never Read by Protocol Logic, Introducing an Unnecessary Oracle Dependency	
Contract	InstitutionalLoanVault.sol
Severity Level	Informational

Description	Description <code>_enterLock()</code> computes and stores <code>_instRuntime.idealCollateralValuation</code> via a <code>_getCollateralValueUSD()</code> oracle call. This field is never read by any business logic — health checks, liquidation, and settlement all ignore it. The oracle call occurs at the critical Fundraising → Lock transition point; if the oracle returns zero at that moment, <code>InvalidOraclePrice</code> is thrown and the transition is blocked for no business reason. <pre> JavaScript // InstitutionalLoanVault - _enterLock() _instRuntime.minimumCollateralRequired = (idealCollateral * totalRaised) / _config.maxBorrowCap; _instRuntime.idealCollateralValuation = _getCollateralValueUSD(_instRuntime.minimumCollateralRequired); // ↑ never read by any protocol logic </pre> Impact: Adds avoidable oracle failure risk at a critical state transition. Wastes gas and storage. No direct fund loss risk.
Recommendation	Remove the <code>idealCollateralValuation</code> assignment. If the snapshot is needed for off-chain monitoring, emit it as part of the <code>VaultLocked</code> event instead.
Status	Fixed in 9d5b8e5 . The <code>idealCollateralValuation</code> field and its associated <code>_getCollateralValueUSD()</code> oracle call have been removed from <code>_enterLock()</code> . The Fundraising → Lock transition no longer carries an unnecessary oracle dependency that could have blocked the state change if the oracle returned zero at that moment.

[I02] PSR Notification Error-Handling Strategy Is Inconsistent Across `_settleProtocolShare` and `sweepProtocolShareToReserve`

Contract	BaseVault.sol, LiquidationAdapter.sol
Severity Level	Informational
Description	Description The two PSR <code>updateAssetsState()</code> call sites use different error-handling strategies: <code>_settleProtocolShare()</code> wraps the call in try/catch and silently emits an event on failure, while <code>sweepProtocolShareToReserve()</code> has no protection and reverts the entire sweep if the PSR call fails <pre> JavaScript // BaseVault - _settleProtocolShare() </pre>

```

try IProtocolShareReserve(psr).updateAssetsState(...) { }
catch (bytes memory reason) {
    emit PSRNotificationFailed(psr, reason); // silently swallowed
}

// LiquidationAdapter - sweepProtocolShareToReserve()
IERC20(collateral).safeTransfer(psr, amount);
IProtocolShareReserve(psr).updateAssetsState(...); // no protection -
reverts on failure

```

Impact:

The inconsistency creates different failure modes: in `_settleProtocolShare`, tokens are transferred but PSR tracking may go out of sync; in `sweepProtocolShareToReserve`, tokens are not transferred but accounting remains consistent. No direct fund loss risk.

Recommendation	Align the error-handling strategy across both call sites. If "never blocking the main flow" is the priority, apply try/catch consistently in both. If "PSR accounting consistency" is the priority, remove try/catch from <code>_settleProtocolShare</code> .
-----------------------	---

Status	Acknowledged. The asymmetric error-handling between <code>_settleProtocolShare</code> and <code>sweepProtocolShareToReserve</code> remains unchanged. The development team's position is that the difference is intentional: in the vault, a PSR notification failure must not block lender redemptions, so the call is wrapped in try/catch and the failure is made observable via <code>PSRNotificationFailed</code> ; in <code>sweepProtocolShareToReserve</code> , the function's sole purpose is to complete the PSR round-trip, so a revert is the correct signal that the operation did not complete and gives governance a clear prompt to retry. This is a reasonable design rationale, and no code change is required.
---------------	--

[I03] The `maxBorrowCap == 0` Check in `_validateVaultConfig` Is Dead Code

Contract	<code>InstitutionalVaultController.sol</code>
-----------------	---

Severity Level	Informational
-----------------------	----------------------

Description	Description The standalone <code>maxBorrowCap == 0</code> check is unreachable: the preceding check <code>minBorrowCap == 0 minBorrowCap > maxBorrowCap</code> already covers every scenario where <code>maxBorrowCap == 0</code> could occur.
--------------------	---

JavaScript

```
// InstitutionalVaultController - _validateVaultConfig()
```

```

    if (vaultConfig.minBorrowCap == 0 || vaultConfig.minBorrowCap >
        vaultConfig.maxBorrowCap) {
        revert InvalidConfig(); // already handles all maxBorrowCap == 0
    cases
    }
    if (vaultConfig.maxBorrowCap == 0) revert InvalidConfig(); //
    unreachable

```

Impact:

Pure dead code. No security impact.

Recommendation Remove the redundant maxBorrowCap == 0 check.

Status Fixed. The redundant maxBorrowCap == 0 check has been removed from _validateVaultConfig. The preceding condition already covers this case — if minBorrowCap > maxBorrowCap and minBorrowCap > 0, then maxBorrowCap cannot be zero without triggering the earlier revert.

[I04] MarginDeposited State Has No Self-Help Exit for the Institution — Collateral Fully Dependent on Governance to Unlock

Contract InstitutionalLoanVault.sol, BaseVault.sol, InstitutionalVaultController.sol

Severity Level **Informational**

Description **Description**
Once the institution deposits the full margin amount in WaitingForMargin, the vault transitions one-way to MarginDeposited. From this point, the only path forward is for a governance-authorized caller to invoke openVault(). If governance does not act — due to ACM approval delays, operational error, or any other reason — the institution's collateral is locked in the contract with no self-help recovery path.

withdrawCollateral does not permit MarginDeposited state:

```

JavaScript
// InstitutionalLoanVault - withdrawCollateral()
function withdrawCollateral(uint256 amount) external onlyPositionHolder
nonReentrant whenNotPaused {
    VaultState s = _checkAndAdvanceState();
    if (s != VaultState.Lock && s != VaultState.Matured && s !=
        VaultState.Failed) {
        revert InvalidState(); // MarginDeposited is not in the allowed
    set

```

```
    }  
    ...  
}
```

closeVault is equally unavailable:

```
JavaScript  
// BaseVault - closeVault()  
function closeVault() external onlyController {  
    VaultState s = _runtime.state;  
    if (s != VaultState.Matured && s != VaultState.Failed && s !=  
VaultState.Liquidated) revert InvalidState();  
    // MarginDeposited is not in the allowed set  
}
```

_checkAndAdvanceState has no branch for MarginDeposited, so the state never auto-advances:

```
JavaScript  
// InstitutionalLoanVault - _checkAndAdvanceState()  
function _checkAndAdvanceState() internal override returns (VaultState) {  
    VaultState s = _runtime.state;  
    if (s == VaultState.Fundraising) { ... }  
    if (s == VaultState.Lock ...) { ... }  
    if (s == VaultState.PendingSettlement) { ... }  
    if (s == VaultState.SettlementDeadlineExceeded ...) { ... }  
    return _runtime.state; // WaitingForMargin / MarginDeposited fall  
through unchanged  
}
```

The only on-chain asset recovery path is governance calling sweep(collateralAsset), but the recipient is treasury, not the institution:

```
JavaScript  
// BaseVault - sweep()  
function sweep(address token) external onlyController {  
    address recipient = IVaultController(vaultController).treasury();  
    IERC20(token).safeTransfer(recipient, amount); // sent to treasury,  
not institution  
}
```

Impact:

The institution loses all self-help control over its deposited collateral while in MarginDeposited state. Both possible recovery paths require governance action: either openVault() → wait for openEndTime → Failed → institution withdraws, or sweep(collateral) → treasury → off-chain coordination to return funds. No direct fund loss risk given the system is explicitly a permissioned, governance-controlled product. This is a defensive design gap that may cause operational friction if governance is delayed or makes an error.

Recommendation Add MarginDeposited to the allowed states in closeVault, enabling governance to formally close an abandoned vault and recover collateral via sweep to treasury for off-chain return to the institution:

JavaScript

```
// BaseVault - closeVault()
if (s != VaultState.Matured && s != VaultState.Failed
    && s != VaultState.Liquidated && s != VaultState.MarginDeposited) {
    revert InvalidState();
}
```

Status Fixed in [b0201f5f](#) and [2aa546d](#). A new ACM-gated cancelVault() function has been added to both the controller and the vault. It can be called from WaitingForMargin or MarginDeposited state, transitions the vault directly to Failed, and refunds any deposited collateral to the current NFT position holder via positionToken.ownerOf().

[I05] error ZeroAddress Declared but Never Referenced; Corresponding Zero-Address Check Missing in initialize()

Contract InstitutionalLoanVault.sol

Severity Level **Informational**

Description InstitutionalLoanVault declares a ZeroAddress custom error, but no revert ZeroAddress() call exists anywhere in the contract. At the same time, initialize() performs no zero-address validation on the positionToken_ parameter — exactly the guard this error was presumably intended for. If positionToken_ is passed as address(0), the stored positionToken will be the zero address, causing all functions protected by the onlyPositionHolder modifier (depositCollateral, withdrawCollateral, claimRaisedFunds) to permanently revert with PositionTokenIdNotSet.

JavaScript

```
// InstitutionalLoanVault - error declaration
error ZeroAddress(); // declared but never referenced

// InstitutionalLoanVault - initialize()
function initialize(
    VaultConfig calldata config_,
    InstitutionalConfig calldata instConfig_,
    RiskConfig calldata riskConfig_,
    IIInstitutionPositionToken positionToken_,
    ...
) external initializer {
    ...
    positionToken = positionToken_; // no zero-address check on
    positionToken_
}
```

Impact:

Under normal deployment flow, createVault always passes a validated non-zero positionToken address, making the realistic trigger probability very low. However, if initialize() were called with a zero address due to a deployment script error, all institution-side vault functions would be permanently disabled with no recovery path. No direct fund loss risk.

Recommendation Apply the already-declared error ZeroAddress to validate positionToken_ at the start of initialize():

JavaScript

```
if (address(positionToken_) == address(0)) revert ZeroAddress();
positionToken = positionToken_;
```

Status Fixed in [ec80e0c](#). The unused error ZeroAddress declaration has been removed from InstitutionalLoanVault. No corresponding zero-address check was added to initialize() — the development team's position is that vault clones are only ever initialised via InstitutionalVaultController.createVault(), which forwards an already-validated positionToken reference, making the guard redundant at the vault level.

[I06] Overdue Liquidation Clears Debt but Vault Ends in Matured, Same as Normal Full Repayment

Contract	InstitutionalLoanVault.sol
Severity Level	Informational
Description	Description liquidateOverdueVault() repays debt and seizes collateral under SettlementDeadlineExceeded but does not move the vault to Liquidated. After debt reaches zero, the next _checkAndAdvanceState() transitions to Matured via _enterMatured(), the same terminal path as ordinary full repayment. _settleProtocolShare() still runs when entering Matured; the gap is chain-visible outcome semantics (state + events), not an obvious skipped settlement. Related Code <pre>JavaScript function liquidateOverdueVault(uint256 repayAmount) external ... { ... uint256 seizeAmount = _executeLiquidation(debt, actualRepay, LiquidationType.DEADLINE); emit OverdueLiquidationExecuted(msg.sender, actualRepay, seizeAmount); } if (s == VaultState.SettlementDeadlineExceeded && _outstandingDebt() == 0) { _enterMatured(); }</pre> Impact: Indexers and dashboards that only read final state == Matured cannot tell “deadline liquidation that cleared debt” from “timely repayment” unless they correlate OverdueLiquidationExecuted, prior states, and logs. No direct fund-loss mechanism was identified from this behavior alone.
Recommendation	If you must distinguish outcomes on-chain, add a dedicated event and/or a dedicated terminal flag or state; otherwise document that Matured is the universal “debt cleared” outcome regardless of path.
Status	Acknowledged. The development team confirms this is intentional behaviour. Matured is the universal terminal state for any path that clears debt to zero, regardless of how it was achieved. Path attribution is already provided by the dedicated OverdueLiquidationExecuted event emitted during the overdue liquidation — off-chain systems that need to distinguish a forced liquidation outcome from a timely repayment can correlate this event against the final Matured state. No code change is required.

